HDOC Day-60

****

Write a C program that reads two strings, s and t. Assume that both strings contain only lowercase characters. Pass the strings to a function named check_anagram() using their addresses. Use pointers to pass the strings between the calling function and check_anagram(). The function check_anagram() must:

- Determine whether s and t contain the same characters with the same frequencies.
- Print "Anagram" if the two strings are valid anagrams.
- Print "Not Anagram" if the two strings are not valid anagrams.
- Treat the order of characters as irrelevant.
- Consider strings of different lengths as not anagrams.
- Use a character-frequency-counting approach to solve the problem.

1. Problem Understanding

The program reads two lowercase strings, s and t, and passes their addresses to check_anagram(). The function first checks whether both strings have the same length. It then uses a 26-element frequency array to count lowercase letters. Characters from s increase their corresponding counts and characters from t decrease them. If every final count is zero, the strings are anagrams; otherwise, they are not anagrams.

2. Standard Test Cases

Test Case	String s	String t	Expected Output
1	listen	silent	Anagram
2	hello	world	Not Anagram
3	anagram	nagaram	Anagram
4	rat	car	Not Anagram
5	aabbcc	abc	Not Anagram

3. Algorithm

Main Algorithm

1. Start.
2. Declare two character arrays s and t.
3. Read string s.
4. Read string t.
5. Pass the addresses of s and t to check_anagram().
6. If the function returns 1, display "Anagram"; otherwise display "Not Anagram".
7. Stop.

Function: check_anagram()

1. Create an integer frequency array freq[26] and initialize all elements to 0.
2. Find the lengths of s and t using pointer traversal.
3. If the lengths are different, return 0.
4. Traverse s and increase freq[s[i] - 'a'] for each character.

5. Traverse t and decrease freq[t[i] - 'a'] for each character.

6. Check all 26 frequency values. If any value is not 0, return 0.

7. Return 1 because both strings contain the same characters with the same frequencies.

4. C Program

```
#include <stdio.h>

int check_anagram(char *s, char *t)
{
  int freq[26] = {0};
  int len1 = 0, len2 = 0, i;

  while(*(s + len1) != '\0')
    len1++;

  while(*(t + len2) != '\0')
    len2++;

  if(len1 != len2)
    return 0;

  for(i = 0; *(s + i) != '\0'; i++)
    freq[*(s + i) - 'a']++;

  for(i = 0; *(t + i) != '\0'; i++)
    freq[*(t + i) - 'a']--;

  for(i = 0; i < 26; i++)
  {
    if(freq[i] != 0)
      return 0;
  }

  return 1;
}

int main()
{
  char s[100], t[100];

  printf("Enter first string: ");
  scanf("%99s", s);

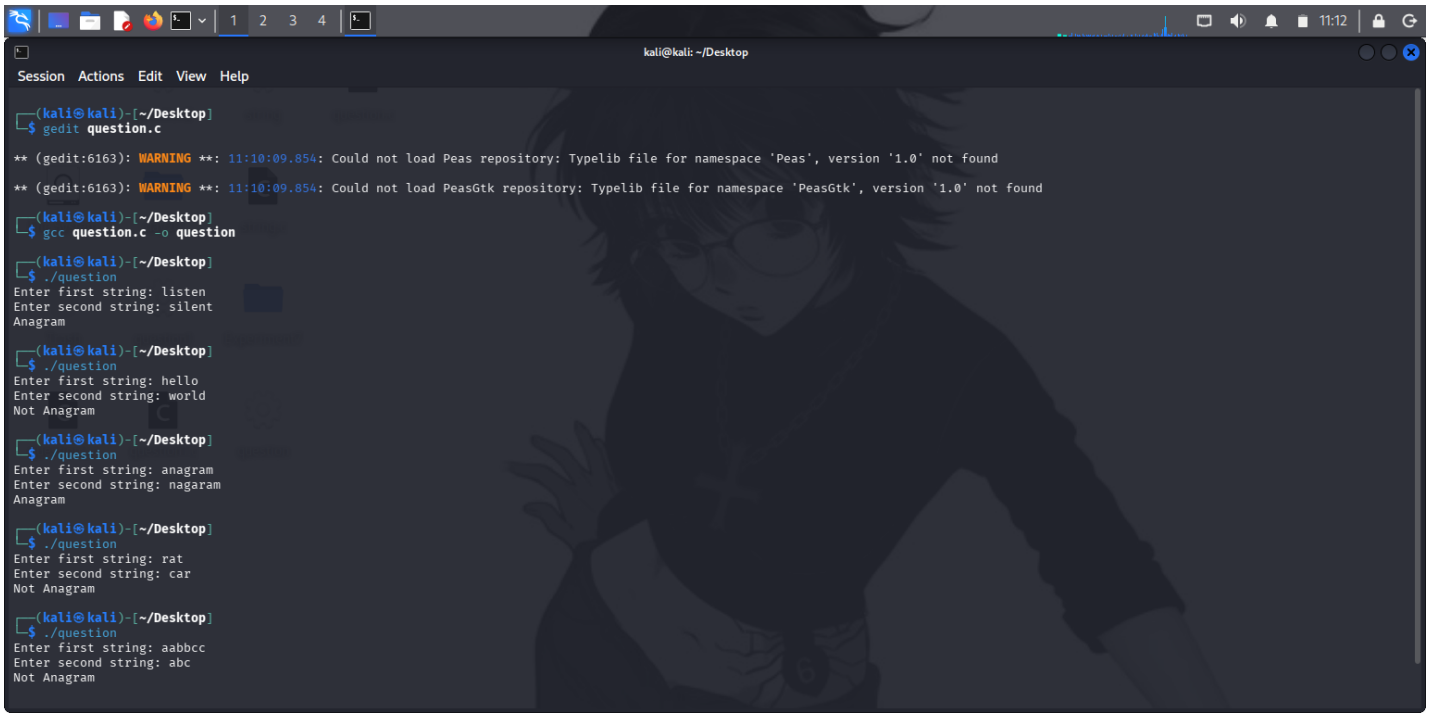
  printf("Enter second string: ");
  scanf("%99s", t);

  if(check_anagram(s, t))
    printf("Anagram\n");
```

```
    else
        printf("Not Anagram\n");

    return 0;
}
```

5. Compilation and Execution Evidence



6. Verification of All Test Cases

Test Case	String s	String t	Actual Result	Status
1	listen	silent	Anagram	Pass
2	hello	world	Not Anagram	Pass
3	anagram	nagaram	Anagram	Pass
4	rat	car	Not Anagram	Pass
5	aabbcc	abc	Not Anagram	Pass